

1. You can only transmit one page of data per connection.
2. You cannot transmit more than the content-length specified when the header is sent.
3. HTTP connections are not conversational. You ask once, and you get once.
4. HTTP connections can stay open for just as long as it takes to transmit the data.

Sure, there are ways around all these limitations. HTTP connections can be re-used, Keep-Alives can be used to keep the connections open for the server to speak up later after the page has been loaded, and Web developers can choose what constitutes ‘one page’ so that they can hack around the protocol to dynamically update pages. But these are all ugly hacks (which have a name—Comet; Google it sometime), and you really can’t have the conversational interface that a raw socket allows.

Enter WebSockets

WebSockets aren’t raw system sockets. Rather, they’re implemented as a special extension on top of HTTP (think of extensions like WebDAV and the like). When you open a WebSocket, it goes through a HTTP handshake—so you can use normal HTTP connection-establishing methods such as authentication or SSL. But after the handshake is over, what you’re left with is a perpetually open connection, where you can pump in and receive any data you like. It doesn’t even have to have a schema.

In short, this means that you can use it just like a pure network socket.

WebSockets are actually the bleeding-edge part of the upcoming set of Web standards. As such, it was only finalised as RFC 6455 as recently as December 2011. It did exist as a number of drafts before. In fact, one such version, Draft 76, was the standard for quite some time, until a gaping security hole was discovered.

With the standard being just two months old, precious few browsers support it. Mozilla Firefox, for one, requires a “Moz” prefix (this should hopefully change with Firefox 11). Chrome has been supporting it since 14.0, but surprisingly, Safari supports only the earlier plagued Draft 76 version of the protocol, as does Opera. And needless to say, Internet Explorer doesn’t support any version of WebSockets. This is slated to change with Windows 8, because the Internet Explorer 10 Technology Preview already includes support for WebSockets.

This is all very well, but how on earth am I supposed to use it? Well, I hope you know Python.

Playing with WebSockets

No, Python isn’t the only way to use WebSockets, but for the purposes of this article, it’s good for a demonstration. Also, it’ll let me introduce another piece of Python technology that’s used much less than it should be—Tornado.

Tornado is an application server written in Python. It

includes its own HTTP server, a template language, and a Web framework that *can* be used like an MVC framework, but is meant to be used in a much easier fashion.

What makes Tornado unique is that it’s asynchronous and real-time. Everyone using FriendFeed retains one connection to them—and the updates are pushed out in real time. That’s easily more than 10,000 connections per server. Yes, Tornado is a FriendFeed technology that was open-sourced after the takeover by Facebook.

Let’s get started. Make sure you’re using Python 2.6 or higher. It’s highly unlikely that Tornado is in your distro’s package repository, though Tornado is included in the PyPI. So you can just use the following command:

```
sudo easy_install tornado
```

And the Tornado package will be installed, along with all dependencies. If you’re the more adventurous type, you could download the source archive from Tornado Web Server (www.tornadoweb.org/), extract it, and use Setuptools to install it the standard way, as follows:

```
tar -xvzf tornado-2.2.tar.gz
cd tornado-2.2
./setup.py build
sudo ./setup.py install
```

If you’re even more adventurous, and want to keep Tornado isolated, you could just extract the archive, and copy the ‘tornado-2.2/tornado’ folder into your project. It’s all good.

Anyway, assuming that Tornado is installed (you can run the Python interpreter and ‘import tornado.web’ without any errors), you’re set to go.

First of all, let us build an application that simply echoes back what we say to it. It’s lifted straight off the Tornado examples, with a few minor edits. Here’s our Python application that’ll run on the server:

```
#!/usr/bin/python2

import tornado.web
import tornado.websocket
import tornado.ioloop

class MainHandler(tornado.web.RequestHandler):
    def get(self):
        self.render("front.html")

class WebSocketHandler(tornado.websocket.WebSocketHandler):
    def on_message(self, message):
        self.write_message(u"Server echoed: " + message)

application = tornado.web.Application([
```